

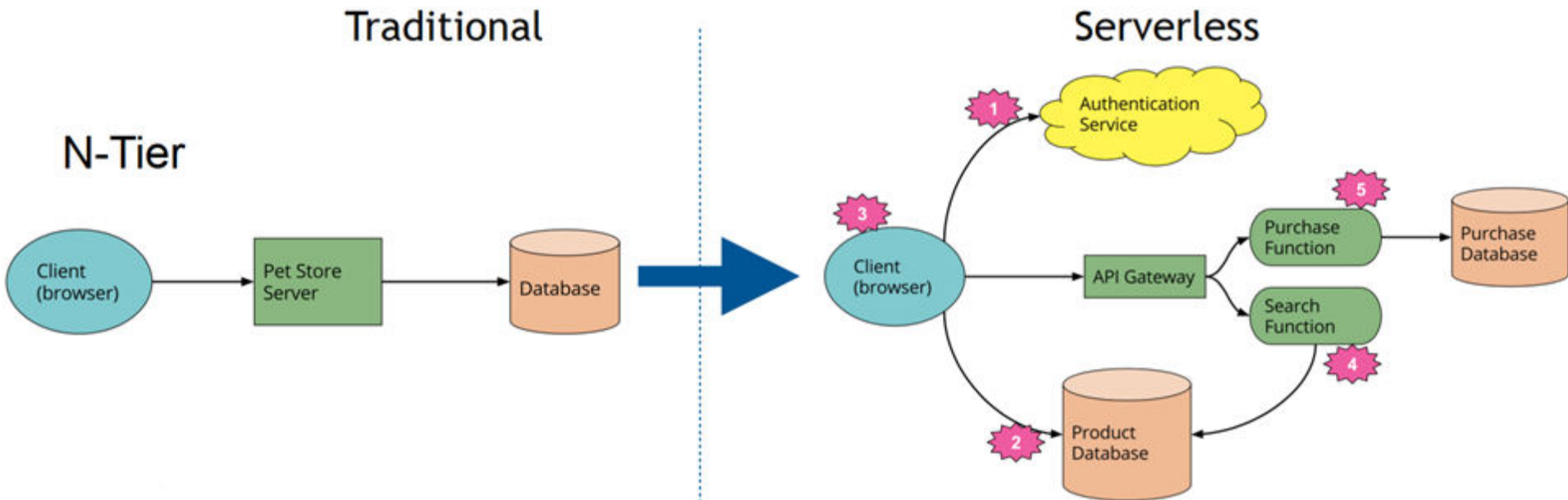
CS 4032 – Web Programming





serverless stack

Making switch to Serverless



Benefits of Serverless



**No servers to provision
or manage**



Scales with usage



Never pay for idle



**Availability and fault
tolerance built in**

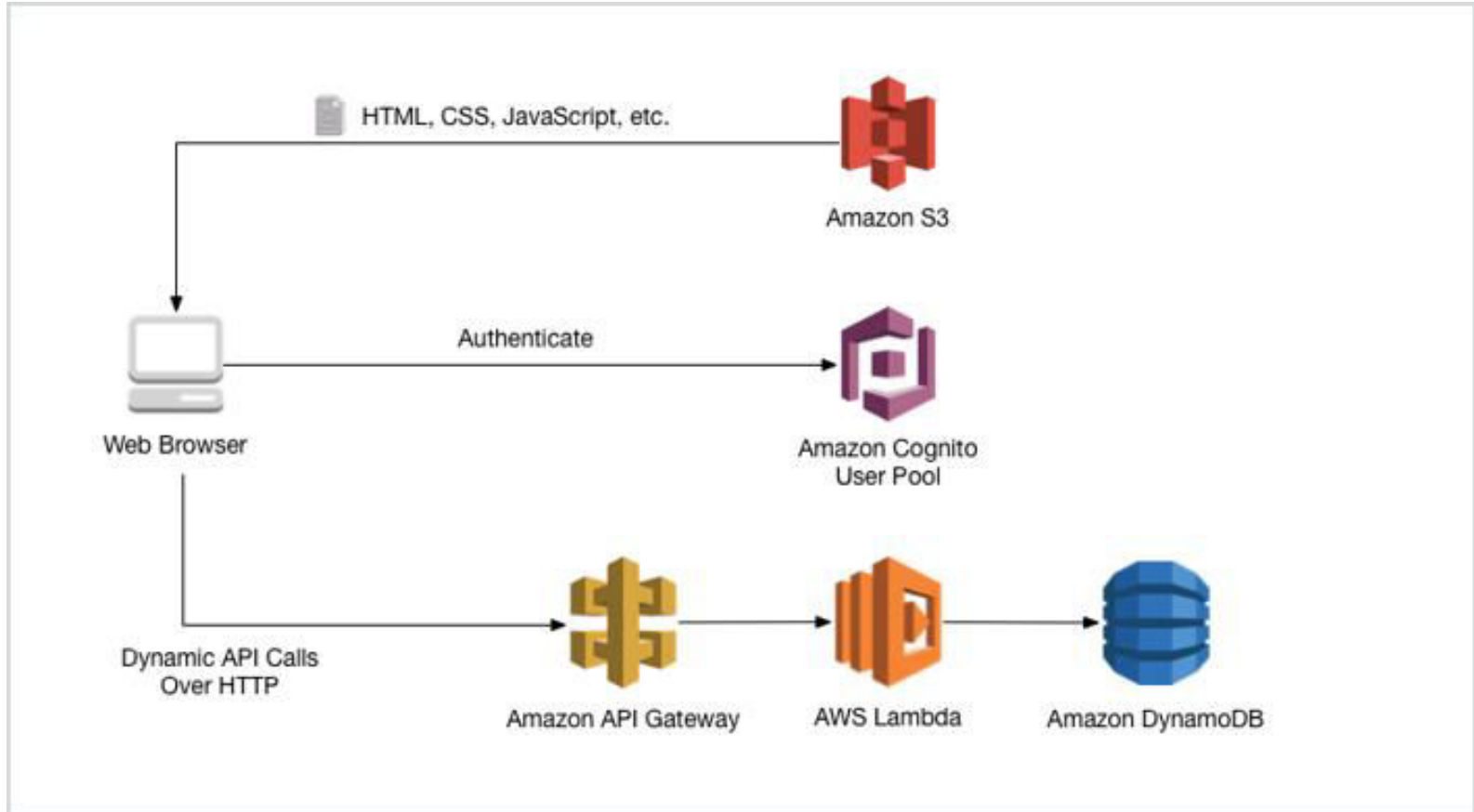
What is Serverless?

- **Serverless** is an execution model where the cloud provider is responsible for executing a piece of code by dynamically allocating the resources.
- **Functions-as-a-Service (FaaS)**
 - The code that is sent to the cloud provider for execution is usually in the form of *a function*
 - FaaS offered by the major cloud providers:
 - AWS: AWS Lambda
 - Microsoft Azure: Azure Functions
 - Google Cloud: Cloud Functions

Microservices

- While transitioning to a serverless world, our application needs to be architected in the form of functions
- Adopt a more microservice based architecture

AWS Serverless Architecture



AWS Lambda

- AWS Lambda (or Lambda for short) is a serverless computing service provided by AWS
- Lambda specifications:
 - A container with a 64-bit Amazon Linux AMI
 - Memory: 128MB - 10240MB, in 1 MB increments
 - Ephemeral disk space: 512MB - 10240MB, in 1 MB increments
 - Max execution duration: 900 seconds (15 minutes)
 - Compressed package size: 50MB
 - Uncompressed package size: 250MB
 - Container image package size: 10GB

Lambda Function

Diagram illustrating the structure of a Lambda Function handler:

```
exports.myHandler = function(event, context, callback) {  
    // Do stuff  
    callback(Error error, Object result);  
}
```

Labels and their corresponding parameters:

- Name of the Lambda**: `myHandler`
- Event Object**: `event`
- Context Object**: `context`
- Callback Function**: `callback`
- Error Object**: `Error error`
- Result Object**: `Object result`

Serverless-Stack.com



Packaging Functions

- Lambda functions need to be packaged and sent to AWS
- A process of compressing the function and all its dependencies and uploading it to an S3 bucket

Execution Model

- A new container is brought up to serve a request
- The cloud provider simply creates multiple instances of the container with our function to serve those requests
- Cannot handle concurrent requests

Stateless Functions

- Functions are typically run inside secure (almost) stateless containers
- A function is invoked in a new container every single time
- Functions running inside a container that is brought up on demand to respond to an event. This is referred to as a *Cold Start*.
- Container may keep up after function has complete execution. If another event is triggered during this time it responds far more quickly and this is typically known as a *Warm Start*.

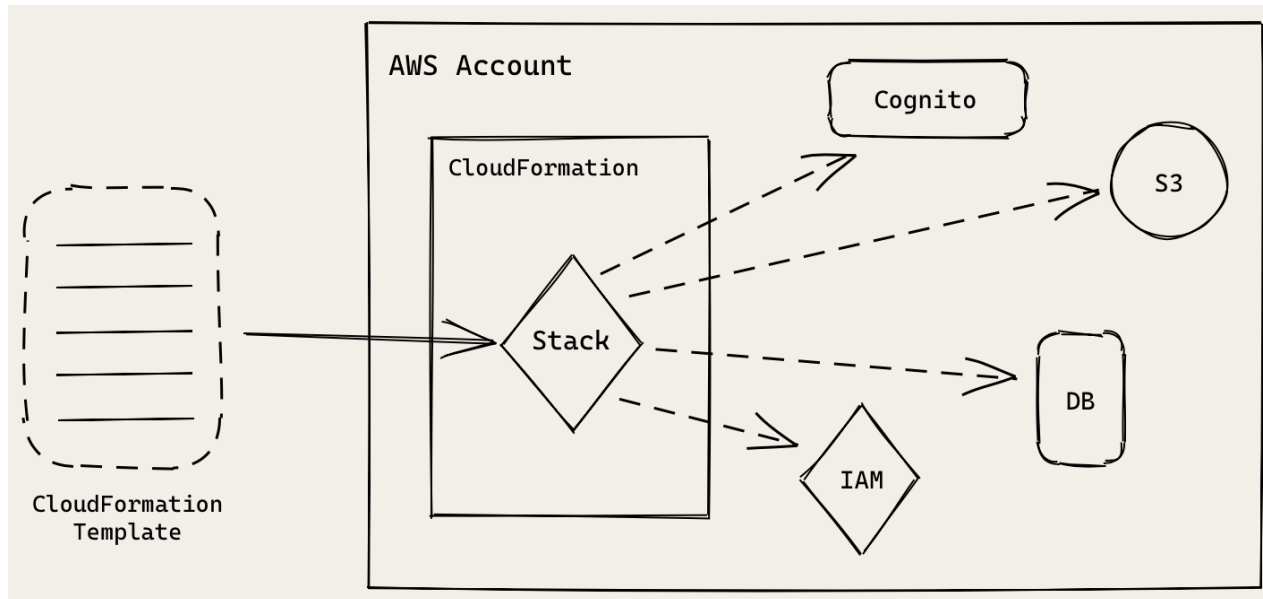
Serverless Stack Framework (SST)

- A framework that makes it easy to build serverless apps
- SST makes it easy to build serverless applications by allowing developers to:
 1. Define their infrastructure using AWS CDK (Cloud Development Kit)
 2. Test their applications live using Live Lambda Development
 3. Set breakpoints and debug in Visual Studio Code
 4. Web-based dashboard to manage your apps
 5. Deploy to multiple environments and regions
 6. Use higher-level constructs designed specifically for serverless apps
 7. Configure Lambda functions with JS and TS, Python, C#, etc.



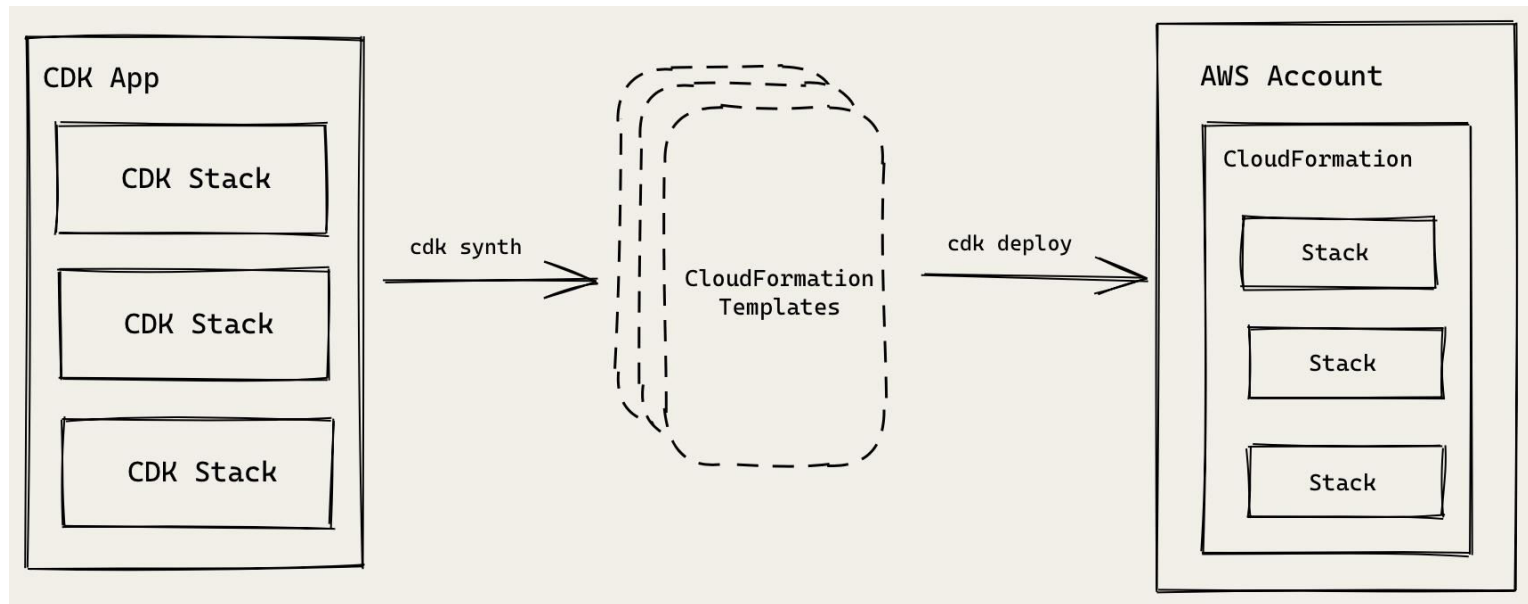
AWS CloudFormation

- SST converts infrastructure code into a CloudFormation template
- CloudFormation is an AWS service that takes a template (written in JSON or YAML), and provisions resources based on that.



AWS CDK (Cloud Development Kit)

- Converts code into a CloudFormation template
- Uses CloudFormation internally



Authentication in Serverless Apps

- Cognito User Pool
 - An AWS service to manage sign up and login functionality for users
 - It will store user's login info
 - It will also be managing user sessions in React app
- Cognito Identity Pool
 - An AWS service to manage access control to our AWS infrastructure
 - This service decides if previously authenticated user has access to the resources he/she is trying to connect



Authentication Role

- Cognito Identity Pool has a set of rules (called an IAM Role) attached to it
- It will list out the resources an authenticated user is allowed to access. These resources are listed using an ID called ARN (Amazon Resource Names).

Authenticated API Requests

- To connect to API:
 - The React client makes a request to API Gateway secured using IAM Auth.
 - API Gateway will check with Identity Pool if the user has authenticated with User Pool.
 - It will use the Auth Role to figure out if this user can access this API.
 - If everything looks good, then our Lambda function is invoked and it will pass in an Identity Pool user id.

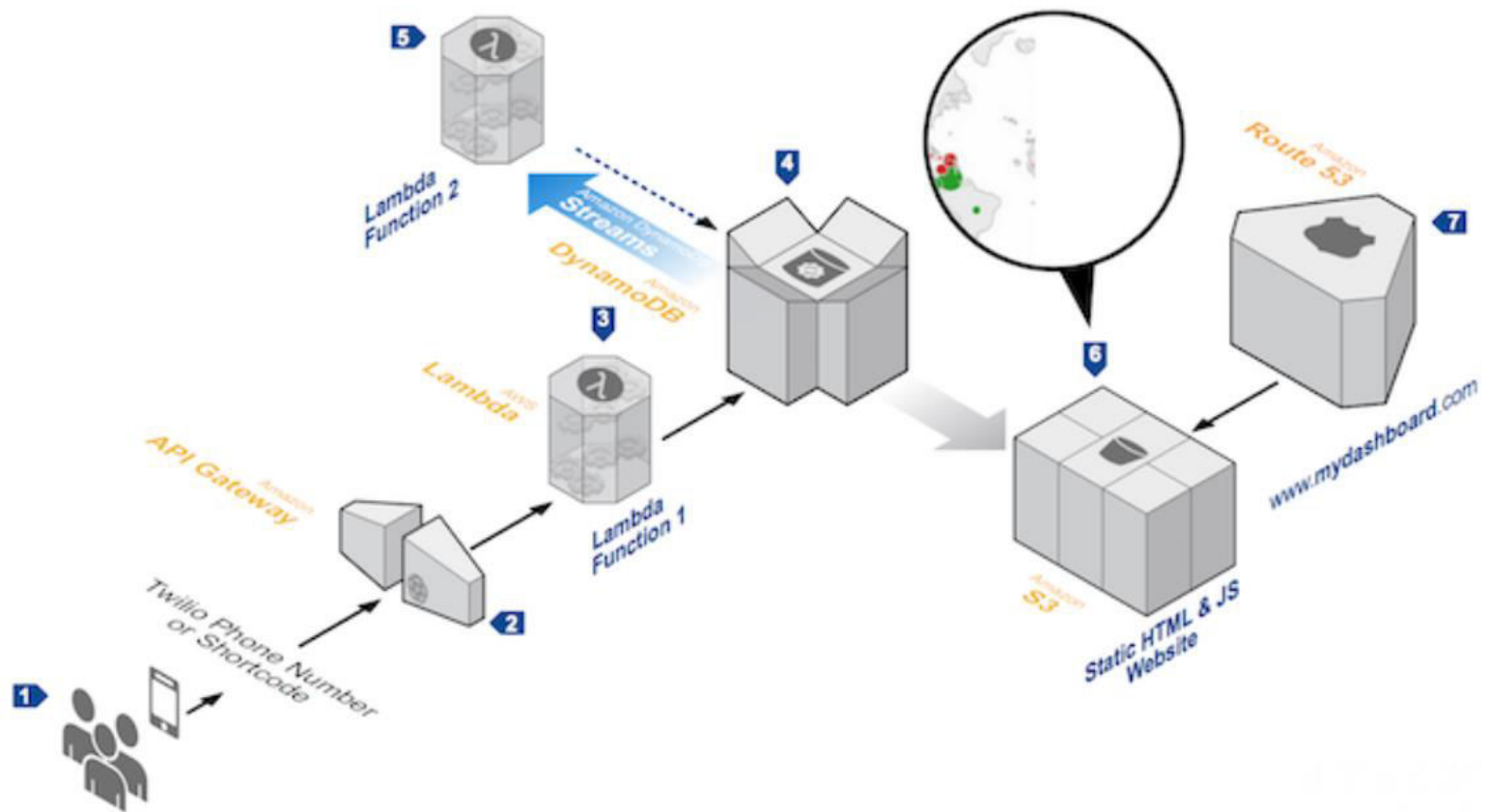


Alternative Authentication Methods

- Use Facebook or Google as an authentication provider
- Manage users and authentication yourself
- You might not be able to manage access control centrally to the S3 bucket (or any other AWS resources in the future).

Serverless in Practice - Architecture

Web applications



Installation

- To use SST you will need:
 - Node.js $\geq$ 10.15.1
 - An AWS account with the AWS CLI configured locally
- Install the AWS Toolkit for Visual Studio Code
 - As an extension
- Install AWS SAM CLI
 - <https://aws.amazon.com/serverless/sam/>

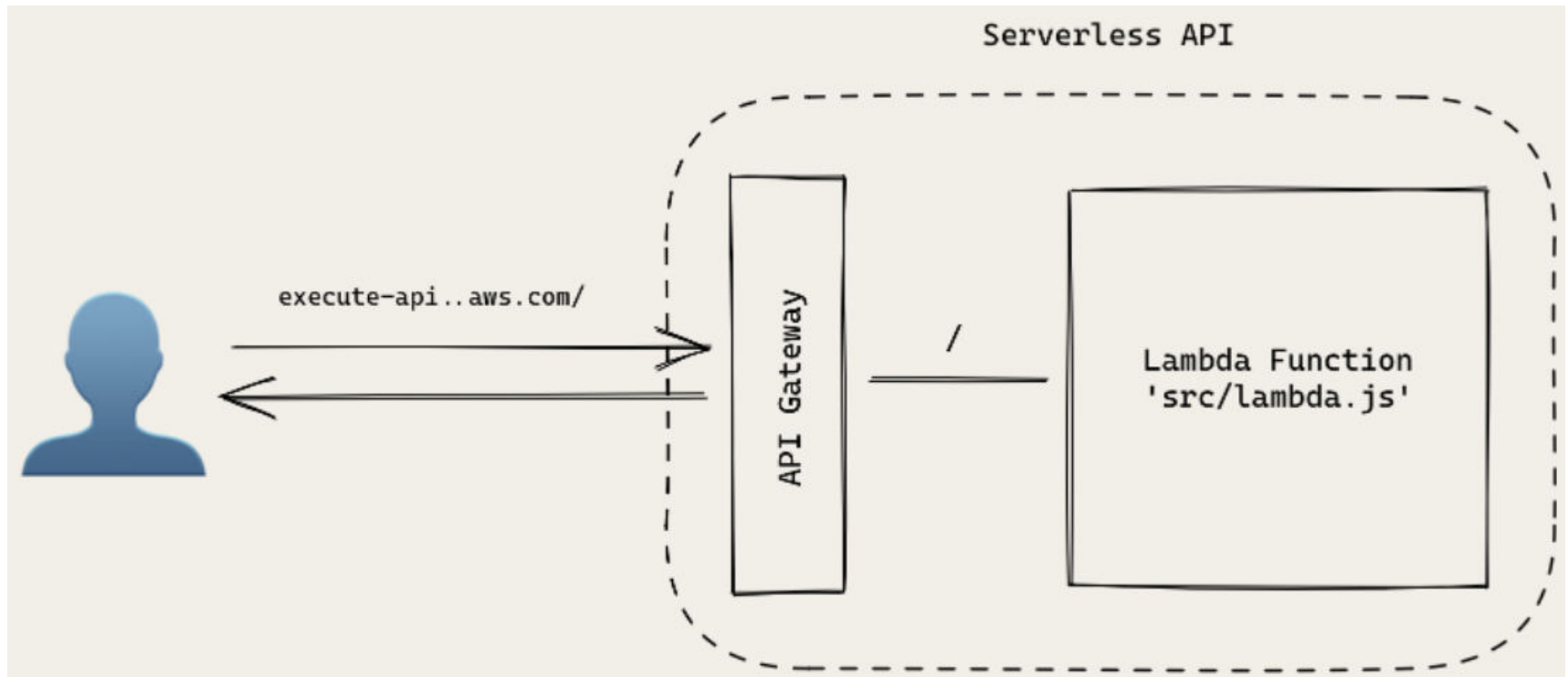
SST CLI

- Installation command
 - `npm install @serverless-stack/cli -g`
 - `npm install create-sst -g`
- Create your first SST app
 - `npm init sst` # Create a new SST app
 - `npm start`

Project layout

- An SST app is made up of three parts:
 - stacks/ — App Infrastructure
 - The code that describes the infrastructure of serverless app is placed in the stacks/directory of the project. SST uses AWS CDK, to create the infrastructure.
 - backend/ — App Code
 - The Lambda function code that's run when the API is invoked is placed in the backend/ directory of the project.
 - frontend/ — directory for frontend React app

Hello World API Architecture



Serverless is Good and Not Good for

- Short-running
- Stateless
- Event-driven
- Microservices
- Mobile Backends
- Bots
- IoT
- Service Integration
- Stream processing
- Long-running
- Stateful
- Number crunching
- Databases
- DL training
- Heavy-duty stream analytics
- Numeric simulation
- Video streaming

Some Resources

- <https://serverless-stack.com/>
- <https://webapp.serverlessworkshops.io/>
- <https://github.com/aws-samples/lambda-refarch-webapp>



Thank you!



National University
Of Computer and Emerging Sciences